



DEPARTAMENTO DE TECNOLOGÍA DIGITAL
CARRERA DE DISEÑO Y DESARROLLO DE SOFTWARE

DESARROLLO DE SOLUCIONES EN LA NUBE

TRABAJO 1 - DOCKER SWARM

INTEGRANTES:

- Santos Zuasnabar, Paulo Nicolas

DOCENTE:

- FARFAN MADARIAGA, JAIME MOSHE

LIMA, MAYO 3

2026 - 1

1. Introducción

TechRetail es una empresa de comercio electrónico que, debido a su crecimiento, comenzó a experimentar problemas de rendimiento, disponibilidad y escalabilidad en su plataforma web. El sistema, inicialmente monolítico, no era capaz de soportar picos de demanda durante campañas comerciales.

Para solucionar estos problemas, se implementó una arquitectura basada en contenedores utilizando Docker y orquestación con Docker Swarm, permitiendo una infraestructura más flexible, escalable y tolerante a fallos.

2. Objetivo

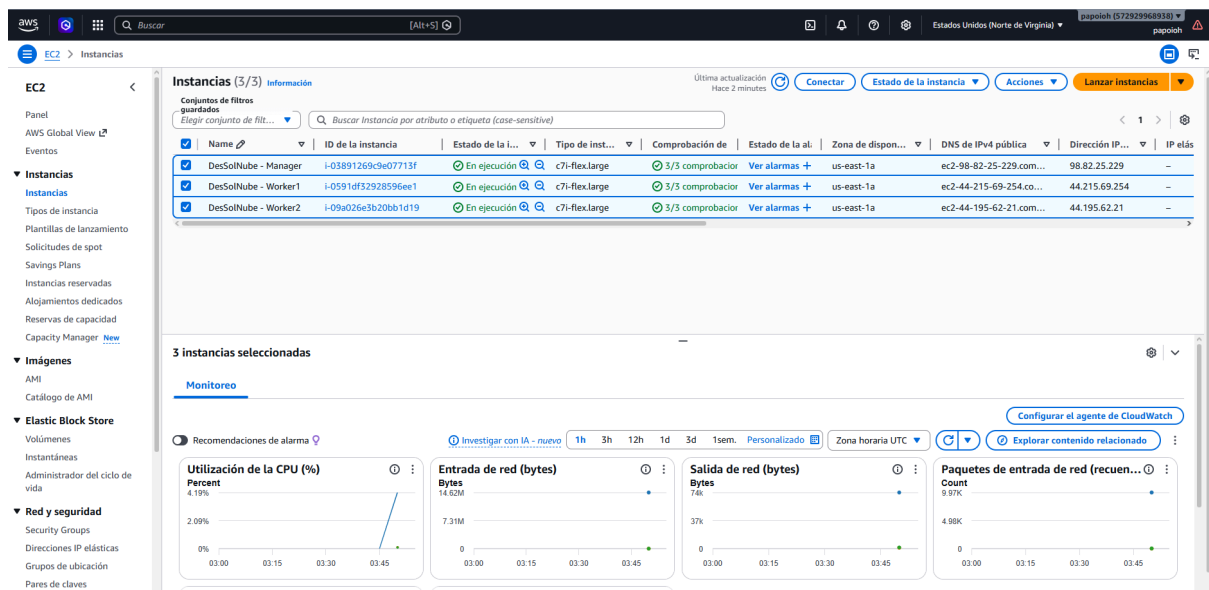
Diseñar e implementar un clúster Docker Swarm que permita:

- Escalar servicios horizontalmente
- Garantizar alta disponibilidad
- Distribuir carga entre múltiples nodos
- Mejorar la gestión de servicios

3. Arquitectura del sistema

La arquitectura implementada está compuesta por:

- **1 nodo Manager:** encargado de la gestión del clúster
- **2 nodos Worker:** encargados de ejecutar los contenedores



Servicios desplegados:

- **Frontend (Nginx):** interfaz web
- **Backend (Node.js):** lógica de negocio
- **Base de datos (MySQL):** persistencia
- **Cache (Redis):** optimización de consultas
- **Visualizer:** monitoreo del clúster

Todos los servicios se comunican mediante una **red overlay**, permitiendo la interacción entre contenedores en diferentes nodos.

4. Implementación

Drive:

<https://drive.google.com/drive/folders/1DGvDosxrVxSEBwYeFQbttmNPfxDGXcZL?usp=sharing>

Github:

<https://github.com/papoi-dvv/DesSolNube-Trabajo1>

4.1 Creación de instancias

Se crearon 3 instancias virtuales en la nube con sistema operativo Ubuntu, configuradas con:

- Docker instalado
- Mismo Security Group
- Conectividad mediante IP privada

4.2 Configuración de seguridad

Inicialmente se utilizó el Security Group por defecto, sin embargo:

“Inicialmente se utilizó el Security Group por defecto, pero en un entorno productivo se recomienda uno dedicado para aislar el tráfico y mejorar la seguridad del clúster.”

Posteriormente se configuraron reglas específicas para:

- Comunicación interna del clúster
- Acceso HTTP (puerto 80)
- Acceso al visualizador (puerto 8080)

Esto generó un token para unir los nodos worker.

4.5 Unión de nodos worker

En los nodos secundarios se ejecutó:

```
docker swarm join --token <TOKEN> <IP_MANAGER>:2377
```

4.6 Verificación del clúster

```
docker node ls
```

```
ubuntu@ip-172-31-4-218:~$ sudo docker node ls
```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
smm9ml4lxfiy2gftyyvui8k2 *	ip-172-31-4-218	Ready	Active	Leader	29.4.2
dcgyl9jqahdb6b4i2lqgn852z	ip-172-31-8-153	Ready	Active		29.4.2
j2qaw3jx7idwmvsvy2j0odg4cx	ip-172-31-10-112	Ready	Active		29.4.2

```
ubuntu@ip-172-31-4-218:~$
```

i-03891269c9e07713f (DesSolNube - Manager)
 PublicIPs: 98.82.25.229 PrivateIPs: 172.31.4.218

Tokens (Manager):

```
ubuntu@ip-172-31-4-218:~$ sudo docker swarm init --advertise-addr 172.31.4.218
Swarm initialized: current node (smm9ml4lxfiy2gftyyvui8k2) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2rjniilz9883xiecqjzaut8gf5tqyycxfqn3if852gpu4xcdyg-b0rh0qwigvybsaca4adlm83zf 172.31.4.218:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
ubuntu@ip-172-31-4-218:~$
```

i-03891269c9e07713f (DesSolNube - Manager)
 PublicIPs: 98.82.25.229 PrivateIPs: 172.31.4.218

(Workers):

```
ubuntu@ip-172-31-10-112:~$ sudo docker swarm join --token SWMTKN-1-2rjniilz9883xiecqjzaut8gf5tqyycxfqn3if852gpu4xcdyg-b0rh0qwigvybsaca4adlm83zf 172.31.4.218:2377
This node joined a swarm as a worker.
ubuntu@ip-172-31-10-112:~$
```

i-0591df32928596ee1 (DesSolNube - Worker1)
 PublicIPs: 44.215.69.254 PrivateIPs: 172.31.10.112

```
ubuntu@ip-172-31-8-153:~$ sudo docker swarm join --token SWMTKN-1-2rjniilz9883xiecqjzaut8gf5tqyycxfqn3if852gpu4xcdyg-b0rh0qwigvybsaca4adlm83zf 172.31.4.218:2377
This node joined a swarm as a worker.
ubuntu@ip-172-31-8-153:~$
```

i-09a026e3b20bb1d19 (DesSolNube - Worker2)
 PublicIPs: 44.195.62.21 PrivateIPs: 172.31.8.153

Se verificó la existencia de:

- 1 nodo líder (manager)
- 2 nodos worker en estado Ready

4.7 Gestión de secretos

Se creó un secreto para la base de datos:

```
docker secret create db_password
```

```
docker secret ls
```

```
ubuntu@ip-172-31-4-218:~$ echo "MiPasswordSegura123" | sudo docker secret create db_password -
gurks7nkb05hh95im12jxkp01
ubuntu@ip-172-31-4-218:~$ sudo docker secret ls
ID                NAME          DRIVER      CREATED          UPDATED
gurks7nkb05hh95im12jxkp01  db_password    local        14 seconds ago   14 seconds ago
ubuntu@ip-172-31-4-218:~$
```

i-03891269c9e07713f (DesSolNube - Manager)

PublicIPs: 98.82.25.229 PrivateIPs: 172.31.4.218

4.8 Despliegue de servicios

Se utilizó un archivo `docker-compose.yml` para definir los servicios y luego:

```
docker stack deploy -c docker-compose.yml techretail
```

```
ubuntu@ip-172-31-4-218:~$ nano docker-compose.yml
ubuntu@ip-172-31-4-218:~$ sudo docker stack deploy -c docker-compose.yml techretail
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network techretail_techretail_net
Creating service techretail_cache
Creating service techretail_visualizer
Creating service techretail_frontend
Creating service techretail_backend
Creating service techretail_database
ubuntu@ip-172-31-4-218:~$ sudo docker stack services techretail
ID                NAME                MODE                REPLICAS            IMAGE                                  PORTS
ah1lwqylfpfl      techretail_backend   replicated           2/2                  node:18-alpine                       *
dzcrumy1gs39      techretail_cache     replicated           1/1                  redis:7-alpine                       *:80->80/tcp
yc38kxw3mik        techretail_database  replicated           0/1                  mysql:8                               *:8080->8080/tcp
prdwzsdz61u        techretail_frontend  replicated           3/3                  nginx:alpine                         *:80->80/tcp
42o6uxm5u0gi       techretail_visualizer replicated           0/1                  dockersamples/visualizer:latest      *:8080->8080/tcp
ubuntu@ip-172-31-4-218:~$
```

i-03891269c9e07713f (DesSolNube - Manager)

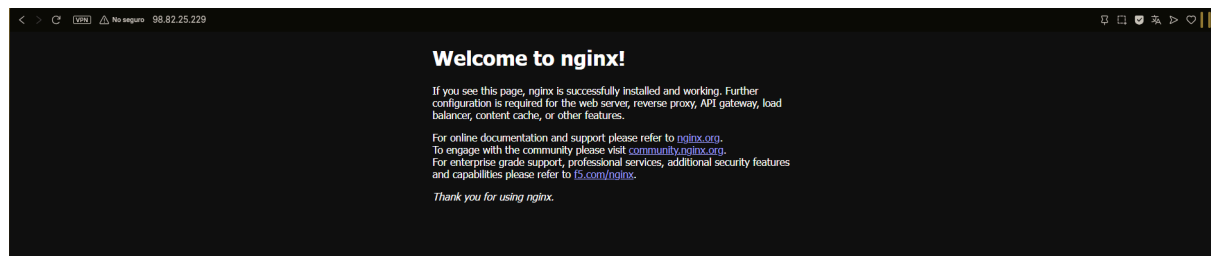
PublicIPs: 98.82.25.229 PrivateIPs: 172.31.4.218

5. Pruebas de funcionamiento

5.1 Acceso web

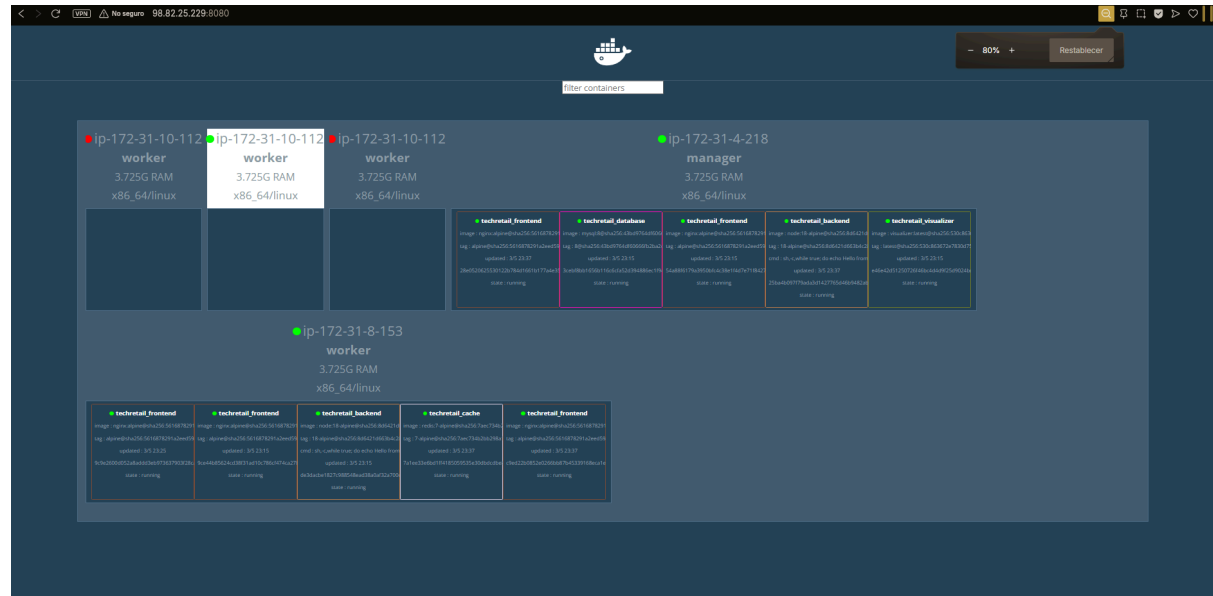
- Frontend:

`http://98.82.25.229`



- Visualizer:

`http://98.82.25.229:8080`



5.2 Escalabilidad

Se realizó el escalado dinámico del frontend:

```
docker service scale techretail_frontend=5
```

Se verificó que:

- Las réplicas aumentaron correctamente
- El servicio no presentó interrupciones

6. Problemas encontrados y soluciones

◆ Problema 1: Error de permisos Docker

- **Error:** permission denied
 - **Solución:** usar **sudo** y agregar usuario al grupo docker
-

◆ **Problema 2: Error de conexión SSH**

- **Causa:** configuración incorrecta del Security Group
 - **Solución:** habilitar puerto 22
-

◆ **Problema 3: Visualizer no accesible**

- **Causa:** puerto 8080 bloqueado
 - **Solución:** configurar correctamente reglas del Security Group
-

◆ **Problema 4: Uso de Security Group por defecto**

- **Solución:** crear uno personalizado para mayor seguridad

7. Resultados obtenidos

- Sistema escalable dinámicamente
- Alta disponibilidad mediante múltiples nodos
- Balanceo automático de carga
- Separación de servicios en microservicios

8. Conclusiones

La implementación de Docker Swarm permitió transformar un sistema monolítico en una arquitectura distribuida, mejorando significativamente la escalabilidad y disponibilidad del sistema.

Se comprobó que Docker Swarm es una herramienta eficiente para la orquestación de contenedores, especialmente en entornos donde se requiere simplicidad y rapidez de implementación.

9. Recomendaciones

- Implementar HTTPS en producción
- Usar balanceadores de carga externos
- Monitoreo avanzado con herramientas como Prometheus
- Automatización con CI/CD